

Prueba Técnica: Full-Stack Engineer (L1)

El objetivo de esta prueba es evaluar cómo aprendes conceptos de alto nivel y cómo resuelves problemas.

Construye un dashboard que monitoree el desempeño de ventas de una empresa de e-commerce.

Entrega: envía un email a erick.naunay@datanalysisgroup.com con el link del repositorio de GitHub al terminar. Tienen hasta el lunes 29 de junio para enviarme el email. Se agendará la entrevista en vivo según la calidad de la solución. La solución debe estar a la altura para avanzar.

Puedes usar la IA para realizar la prueba técnica. Sin embargo, es obligatorio que entiendas todo el código, arquitectura, decisiones técnicas y que en la entrevista defiendas la solución con todo lo que presentas.

NO ME HAGAN PERDER EL TIEMPO, en la entrevista en vivo no podrán usar IA.

Si no pueden defender los conceptos, no envíen su solución.

1. Stack obligatorio

Capa	Requerimiento
Frontend	Next.js (TypeScript)
Backend	Node.js + Express (TypeScript), arquitectura hexagonal
Base de datos	PostgreSQL con capas raw / clean / gold
ORM	Requerido: Prisma o TypeORM
Modelado	Esquema estrella en gold
Infra	Docker Compose con 3 servicios: frontend, backend, db

2. Dataset

Brazilian E-Commerce Public Dataset by Olist (~100k órdenes; tablas: orders, order_items, products, customers, payments, etc.).

- Descarga los CSV desde el mirror público de GitHub y cárgalos en raw.*
- No se permite reemplazar el dataset por uno inventado (*solo se acepta mock data en tests unitarios*).

3. KPIs obligatorios

El dashboard debe filtrar por **rango de fechas** (sobre order_purchase_timestamp) más **2 filtros adicionales** (ej. order_status, product_category_name, customer_state).

#	KPI	Definición
1	GMV	Σ item_price de order_items en el rango. El shipping se muestra aparte.
2	Revenue (Paid)	Σ payment_value de órdenes con pago. Múltiples pagos por orden se suman.

#	KPI	Definición
3	Orders	COUNT(DISTINCT order_id) en el rango.
4	AOV	Revenue / Orders (0 si Orders = 0).
5	Items per Order	COUNT(order_item_id) / Orders.
6	Cancellation Rate	canceled_orders / total_orders. Documenta si incluyes 'unavailable'.
7	On-Time Delivery	Sobre entregas: delivered_customer_date ≤ estimated_delivery_date.
8	Top Products	Ranking Top N por GMV y por Revenue (dos rankings).
9	Revenue Trend	Serie temporal (día o semana) de Revenue y Orders.

4. Modelado de datos (PostgreSQL)

Capas

Esquema	Contenido
raw	Tablas que reflejan los CSV tal cual (tipos adecuados, mínimos constraints).
clean	Datos conformados: tipos correctos, null-handling, normalización y deduplicación.
gold	Esquema estrella para analítica.

Esquema estrella (gold)

Grano de la fact: 1 fila por ítem de orden (order_id + order_item_id).

Tabla	Tipo	Atributos / medidas
gold.fact_sales	Hechos	item_price, freight_value, payment_value_allocated, flags is_delivered / is_canceled / is_on_time, FKs a dimensiones.
gold.dim_date	Dimensión	Calendario.
gold.dim_customer	Dimensión	customer_id + geo (state, city).
gold.dim_product	Dimensión	product_id + category.
gold.dim_order	Dimensión	order_id + status + timestamps relevantes.

5. Backend (hexagonal)

Carpeta	Responsabilidad
src/domain	Entidades / Value Objects + contratos de repositorios (ports).
src/application	Use cases: GetKpis, GetRevenueTrend, GetTopProducts.
src/infrastructure	Implementaciones: repos ORM, DB client, migrations, seed.
src/adapters/http	Controllers / routes + DTOs + validación.

Flujo: HTTP → use case → ports (interfaces) → infra implementa con ORM-SQL.

Endpoints REST mínimos

GET /health

GET /kpis?from=YYYY-MM-DD&to=YYYY-MM-DD&...filtros

GET /trend/revenue?from=...&to=...&grain=day|week&...filtros

GET /rankings/products?from=...&to=...&metric=gmv|revenue&limit=10

Validación: fechas obligatorias, rango válido, límite máximo, etc.

El backend NO puede consultar raw ni clean. Todas las queries del API salen de gold.fact_sales como driving table. Se permite JOIN a dimensiones para atributos.

6. Frontend (Next.js)

Vista	Contenido
Overview	Cards de GMV, Revenue, Orders, AOV, IPO, Cancel Rate, On-Time + gráfico de tendencia (Revenue y Orders).
Rankings	Tabla de Top productos con opción de cambiar métrica (GMV / Revenue).
Filtros globales	Date range + 2 filtros extra (estado del cliente, categoría, status de orden).

- TypeScript con manejo de estados de loading y error.
- UI clara y ordenada.
- Consume el API del backend **nunca query directo a la DB.**

7. Docker y ETL

Docker Compose con 3 servicios (db, backend, frontend), variables de entorno, healthchecks y volumen para persistencia de Postgres.

Comando automático / one-shot que:

- Crea schemas y tablas.
- Carga raw → transforma a clean → construye gold (dims + fact).

Implementación: SQL migrations para transforms + orquestación en Node (recomendado).

8. Calidad y entregables

Calidad

- Al menos 3 tests unitarios (dominio o use cases); 1 test de integración del API.
- Manejo de errores con HTTP codes consistentes. ESLint/Prettier.

Repositorio Git

- /frontend (Next.js), /backend (Express), docker-compose.yml.

README

- Setup, comandos y URLs (front / back).
- Arquitectura (diagrama simple vale) y modelo estrella (tabla con dims/fact y grano).
- Definición de KPIs implementados y regla de asignación de payment_value a nivel ítem.
- Tablas cargadas en raw, reglas de limpieza (clean), decisiones técnicas y tradeoffs.